

Infrastructure as Code & Terraform Training

Session Guide — Terraform Basics through Advanced & Cloud Templates

Duration: 3 hours · 9:00 PM – 12:00 AM

Session Agenda

Time	Block	Content
9:00 – 9:10	Kickoff	Intro, objectives, tools check (Terraform + AWS CLI installed)
9:10 – 9:30	Part 1.1	IaC concepts and workflow
9:30 – 9:55	Part 1.2	Terraform providers
9:55 – 10:25	Part 1.3	Resource provisioning (live demo)
10:25 – 10:45	Part 1.4	Local state management
10:45 – 10:55	Break	—
10:55 – 11:20	Part 2.1	Variables, outputs, and workspaces
11:20 – 11:40	Part 2.2	Remote state & state locking
11:40 – 11:55	Part 2.3	AWS CloudFormation & Terraform modules
11:55 – 12:00	Part 2.4	Terraform vs native cloud templates — wrap-up & Q&A

Part 1 — Infrastructure as Code: Terraform Basics

1.1 IaC Concepts and Workflow

Concepts to cover

- **What is IaC:** managing and provisioning infrastructure through machine-readable definition files instead of manual configuration or interactive tools.
- **Why it matters:** consistency across environments, version control for infra (Git), repeatability, faster provisioning, disaster recovery, peer review via pull requests.
- **Declarative vs Imperative:** Terraform is declarative — you describe the desired end state, not the steps to get there. Contrast with imperative scripting (Bash/Ansible in procedural mode) where you specify the exact sequence of actions.
- **Terraform's core workflow:** Write → Init → Plan → Apply → Destroy.

Workflow diagram (verbal): Write HCL config → `terraform init` (download providers/modules) → `terraform plan` (preview changes) → `terraform apply` (create/update infra) → `terraform destroy` (tear down).

```
# Core commands to type live
terraform init
terraform validate
terraform plan
terraform apply
terraform destroy
```

Talking point: Emphasize idempotency — running `terraform apply` repeatedly with no config changes results in "No changes" rather than re-creating resources.

1.2 Terraform Providers

Concepts to cover

- A **provider** is a plugin that lets Terraform talk to an API (AWS, Azure, GCP, Kubernetes, etc.).
- Providers are declared in a `terraform` block and configured in a `provider` block.
- Provider versions are locked via `.terraform.lock.hcl`, generated on `terraform init`.
- The Terraform Registry (registry.terraform.io) hosts official, partner, and community providers.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region  = "us-east-1"
  profile = "default"
}
```

```
# Commands
terraform init      # downloads provider plugins
terraform providers # shows provider tree
terraform version   # shows Terraform + provider versions
```

Talking point: Multiple provider instances (aliasing) let you manage multi-region or multi-account setups — e.g. `provider "aws" { alias = "us_west" }`.

1.3 Resource Provisioning

Concepts to cover

- A **resource block** is the fundamental unit — it declares an infrastructure object (VM, bucket, network, etc.).
- Syntax: `resource "<provider_type>" "<local_name>" { ... }`
- Resources reference each other via attribute interpolation, creating an implicit dependency graph.
- Terraform builds a dependency graph and provisions resources in the correct order automatically.

```
# Example: EC2 instance + Security Group + S3 bucket
resource "aws_security_group" "web_sg" {
  name        = "web-sg"
  description = "Allow HTTP/SSH"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  ami = "ami-0abcdef1234567890"
```

```
instance_type           = "t3.micro"
vpc_security_group_ids = [aws_security_group.web_sg.id]

tags = {
  Name = "training-demo-instance"
}

resource "aws_s3_bucket" "demo_bucket" {
  bucket = "iac-training-demo-bucket-001"
}
```

```
# Live-demo commands
terraform plan -out=tfplan
terraform apply tfplan
terraform show
terraform destroy -target=aws_instance.web  # destroy a single resource
```

Talking point: Show `terraform plan` output and walk through the +/-/~ symbols (create/destroy/update-in-place) before applying.

1.4 Local State Management

Concepts to cover

- Terraform tracks real-world resource IDs and metadata in a **state file**: `terraform.tfstate` (JSON format).
- State is how Terraform maps your config to actual infrastructure and calculates diffs on each plan.
- By default, state is stored **locally** on disk — fine for solo learning/demo, risky for teams (no locking, easy to lose, may contain secrets in plaintext).
- A backup file `terraform.tfstate.backup` is auto-created before each state-changing operation.

```
# State inspection commands
terraform state list           # list all resources in state
terraform state show aws_instance.web  # show one resource's attributes
terraform show                 # human-readable full state
terraform state mv             # rename/move a resource in state
terraform state rm aws_instance.web  # remove from state (not from cloud)
terraform refresh              # sync state with real infra
```

Risk to flag: local state has no locking — two people running `apply` at once can corrupt state or cause conflicting changes. This motivates remote state (Part 2.2).

Part 2 — Terraform Advanced & Cloud Templates

2.1 Terraform Variables, Outputs, and Workspaces

Variables

- Declared in `variables.tf`, referenced as `var.name`.
- Support types: string, number, bool, list, map, object; support defaults and validation blocks.
- Values supplied via: default in block, `.tfvars` files, `-var` CLI flag, or environment variables (`TF_VAR_name`).

```
variable "instance_type" {
  type      = string
  default   = "t3.micro"
  description = "EC2 instance size"
}

variable "environment" {
  type      = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

# terraform.tfvars
instance_type = "t3.small"
environment   = "staging"
```

```
# CLI usage
terraform plan -var="environment=prod"
terraform apply -var-file="prod.tfvars"
```

Outputs

- Declared in `outputs.tf`; expose values after apply (useful for chaining modules or feeding CI/CD pipelines).

```
output "instance_public_ip" {
  value      = aws_instance.web.public_ip
  description = "Public IP of the web instance"
}
```

```
terraform output          # show all outputs
terraform output instance_public_ip # show one output
```

Workspaces

- Workspaces let you maintain multiple state files for the same config (e.g. dev/staging/prod) without duplicating code.
- Default workspace is named `default`.

```
terraform workspace list
terraform workspace new dev
terraform workspace select prod
terraform workspace show
```

Talking point: Workspaces are good for lightweight environment separation, but for larger orgs, separate state files/directories per environment are often preferred over workspaces — worth a quick discussion of trade-offs.

2.2 Remote State & State Locking

Concepts to cover

- Remote state stores `terraform.tfstate` in a shared backend (S3, Azure Blob, GCS, Terraform Cloud) so teams collaborate safely.
- **State locking** prevents concurrent `apply` operations from corrupting state — typically implemented via DynamoDB (AWS) using conditional writes.
- Remote backends also give you encryption at rest, versioning, and audit history.

```
terraform {
  backend "s3" {
    bucket     = "my-terraform-state-bucket"
    key        = "envs/prod/terraform.tfstate"
    region     = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt    = true
  }
}
```

```
# Commands
terraform init -migrate-state    # migrate local state to remote backend
terraform init -reconfigure      # reinitialize backend config
terraform force-unlock          # manually release a stuck lock (use with caution)
```

Demo idea: Open two terminals, run `terraform apply` in both against the same remote backend — show the second one blocked with a "state locked" error, then release and retry.

2.3 AWS CloudFormation & Terraform Modules

Terraform Modules

- A **module** is a reusable, self-contained package of Terraform config (own variables, resources, outputs).
- The root module can call child modules to avoid duplicating code across environments/projects.

```
# Directory structure
modules/
  ec2-instance/
    main.tf
    variables.tf
    outputs.tf
main.tf # root module

# Calling a module from root main.tf
module "web_server" {
  source      = "../modules/ec2-instance"
  instance_type = "t3.micro"
  environment  = "staging"
}
```

```
terraform get      # download/update modules
terraform init     # also initializes modules
terraform plan
terraform apply
```

AWS CloudFormation

- AWS-native IaC service; templates written in JSON or YAML, organized into **stacks**.
- Template sections: `Parameters`, `Resources`, `Outputs`, `Mappings`, `Conditions`.

```
# cf-template.yaml
AWSTemplateFormatVersion: "2010-09-09"
Parameters:
  InstanceTypeParam:
    Type: String
    Default: t3.micro
Resources:
  WebServer:
    Type: AWS::EC2::Instance
    Properties:
      InstanceType: !Ref InstanceTypeParam
      ImageId: ami-0abcdef1234567890
Outputs:
  InstanceId:
    Value: !Ref WebServer
```

```
# CloudFormation CLI commands
aws cloudformation create-stack --stack-name demo-stack --template-body file://cf-template.yaml
aws cloudformation deploy --template-file cf-template.yaml --stack-name demo-stack
aws cloudformation describe-stacks --stack-name demo-stack
aws cloudformation delete-stack --stack-name demo-stack
```

2.4 Terraform vs Native Cloud Templates

Dimension	Terraform	Native Templates (CloudFormation/ARM)
Cloud scope	Multi-cloud (AWS, Azure, GCP, K8s, SaaS providers)	Single-cloud (vendor-specific)
Language	HCL (purpose-built, relatively concise)	JSON/YAML (verbose, more boilerplate)
State management	Explicit state file (local or remote)	State managed internally by the cloud provider
Modules/reuse	First-class modules + public Registry	Nested stacks / SAM (more limited ecosystem)
Drift detection	Via <code>terraform plan</code> /refresh	Native drift detection (CloudFormation)
Vendor lock-in	Low — portable across clouds	High — tied to one provider's ecosystem
Community/ecosystem	Very large, active registry & community modules	Smaller, provider-curated

Closing discussion point: Use Terraform when multi-cloud portability or a consistent workflow across providers matters; use native templates when you're fully committed to one cloud and want the tightest first-party integration (e.g. CloudFormation's native rollback and drift detection).

Quick Command Cheat Sheet

```
## Core workflow
terraform init
terraform validate
terraform plan
terraform apply
terraform destroy

## Providers
terraform providers
terraform version

## State
terraform state list
terraform state show
terraform state mv
terraform state rm
terraform refresh
terraform show

## Variables & outputs
terraform plan -var="key=value"
terraform apply -var-file="env.tfvars"
terraform output
terraform output

## Workspaces
terraform workspace list
terraform workspace new
terraform workspace select

## Remote backend
terraform init -migrate-state
terraform init -reconfigure
terraform force-unlock

## Modules
terraform get

## AWS CloudFormation
aws cloudformation create-stack --stack-name --template-body file://
aws cloudformation deploy --template-file --stack-name
aws cloudformation describe-stacks --stack-name
aws cloudformation delete-stack --stack-name
```